

Security Remediation Report

Project: VERACISM – Verifiable Academic Records for International School Manila **Prepared by:** Development Team

Date: March 27, 2026

Classification: Internal — Restricted

Status: Remediated

Executive Summary

A security assessment of the VERACISM platform identified five vulnerabilities across three risk categories: unauthorized cross-tenant data access (BOLA), sensitive data exposure, and insufficient input validation on file uploads. All findings have been reviewed, validated against the source code, and fully remediated as of this report date. No data breach or unauthorized access has been confirmed; the vulnerabilities were identified and corrected prior to exploitation.

Findings Overview

ID	Title	Severity	Status
VUL-01	Unbounded API Client Data Response	Low	Remediated
VUL-02	Broken Object Level Authorization (BOLA) on API Client Records	Medium–High	Remediated
VUL-03	Insufficient File Type Validation on Document Upload	Low–Medium	Remediated
VUL-04	Broken Object Level Authorization (BOLA) on Report Records	Medium–High	Remediated
VUL-05	Missing Tenant Isolation on Encrypted File View and Download	Medium–High	Remediated

Detailed Findings and Remediation

VUL-01 — Unbounded API Client Data Response

Severity: Low **Affected Endpoint:** `GET /api/apiclient` **Affected Component:** API Client Controller, API Client Repository

Description: The API client listing endpoint returned all records for a tenant in a single response with no pagination enforced. While no secret keys were exposed in the response body (key material is returned only at creation time and stored as a one-way hash), the absence of pagination created an uncontrolled data dump pattern that is contrary to security best practices and increases surface area for denial-of-service scenarios.

Risk: A tenant with a large number of API client records could inadvertently trigger an oversized response, degrading platform performance. The absence of a page size cap also made the endpoint inconsistent with the platform's intended data access model.

Remediation Applied:

- Implemented server-side pagination with configurable `page` and `pageSize` query parameters (defaults: page 1, page size 20; maximum enforced cap: 100 records per page)
- Updated the database query layer to use `OFFSET/FETCH` for efficient row windowing
- Updated the response envelope to include metadata: { `data`, `total`, `page`, `pageSize` }
- Updated all frontend consumers to handle the new response shape

Files Modified:

- `VERACISMAPI/Services/ApiClientRepository.cs`
 - `VERACISMAPI/Controllers/ApiClientsController.cs`
 - `VERACISM/frontend/src/app/services/file.service.ts`
 - `VERACISM/frontend/src/app/pages/admin/settings/apikeys/api-keys-settings.ts`
 - `VERACISM/frontend/src/app/pages/tenant/users/tenant-users.ts`
-

VUL-02 — Broken Object Level Authorization (BOLA) on API Client Records

Severity: Medium–High **Affected Endpoint:** `GET /api/apiclient/{id}`, `PUT /api/apiclient/{id}`, `PATCH /api/apiclient/{id}/status`, `DELETE /api/apiclient/{id}` **Affected Component:** API Client Controller

Description: The `GET /api/apiclient/{id}` endpoint fetched any API client record by its primary key and returned it to any authenticated user without verifying that the requested record belonged to the caller's tenant. A tenant user from Institution A could construct a request for a resource ID belonging to Institution B and receive its details — including the client name, assigned scopes, and key prefix — without authorization.

The same authorization gap was present in the update, status-change, and delete endpoints. While those operations had a secondary safeguard in the database layer (a `WHERE tenant_id = @TenantId` clause that would silently fail cross-tenant writes), the read path had no such protection and actively leaked data.

Risk: Multi-tenancy is a foundational security property of the VERACISM platform. Any cross-tenant data access violates tenant isolation guarantees and could expose institutional metadata to unauthorized parties. Sequential integer primary keys made record ID enumeration trivial.

Remediation Applied:

- Surfaced the `isAdmin` flag from the existing tenant-resolution helper method
- Added an explicit ownership check on the `GET`, `PUT`, `PATCH`, and `DELETE` endpoints: non-admin users receive `HTTP 403 Forbidden` when requesting a record that does not belong to their tenant
- Admin users retain cross-tenant access as required by the platform design

Files Modified:

- `VERACISMAPI/Controllers/ApiClientsController.cs`
-

VUL-03 — Insufficient File Type Validation on Document Upload

Severity: Low–Medium **Affected Endpoints:** `POST /api/scan/scan-upload`, `POST /api/v1/scan-upload`

Affected Component: Scan Controller, V1 Controller, File Upload UI

Description: Both document upload endpoints accepted any file type without validating the file extension against an allowlist of legitimate academic document formats. Executables (`.exe`), scripts (`.php`, `.sh`), and other categorically inappropriate file types were accepted and passed through to the threat-scanning pipeline. While the scanning pipeline (ClamAV + YARA) correctly flagged and blocked the storage of malicious files, the absence of an early type-check created unnecessary processing exposure and relied entirely on downstream controls as the sole line of defense.

The client-supplied `Content-Type` header was also stored in the database without validation, enabling content-type mismatch scenarios.

Risk: If threat-scanning services become unavailable (ClamAV/YARA errors are handled gracefully with fallback), files could potentially be classified as safe and stored without adequate inspection. Processing resources are also wasted scanning files that are categorically outside the platform's intended use.

Remediation Applied:

- Created a shared `FileValidationHelper` class with a strict allowlist of 18 document-appropriate file extensions: PDF, common image formats (JPG, PNG, GIF, WebP), Microsoft Office formats (DOC, DOCX, XLS, XLSX, PPT, PPTX), OpenDocument formats (ODT, ODS, ODP), plain text, and CSV
- Validation is applied as the first check in both upload handlers — before file size validation and before the scanning pipeline is engaged. Unsupported file types are rejected per-file with an `unsupported_file_type` reason code; other files in the same batch continue processing normally
- Updated the frontend file picker to enforce the same allowlist via the `accept` attribute, preventing unsupported files from being selected in the browser UI
- Added client-side pre-validation in the upload widget as a second layer before any API call is made
- Per-file rejection feedback is displayed inline; batch summary messaging distinguishes between partial success and full rejection

Files Modified:

- `VERACISMAPI/Helpers/FileValidationHelper.cs` (*new file*)
- `VERACISMAPI/Controllers/ScanController.cs`
- `VERACISMAPI/Controllers/V1Controller.cs`
- `VERACISM/frontend/src/app/pages/tenant/files/components/fileuploadwidget.ts`
- `VERACISM/frontend/src/app/pages/tenant/files/tenant-files.ts`
- `VERACISM/frontend/src/app/services/file.service.ts`

VUL-04 — Broken Object Level Authorization (BOLA) on Report Records

Severity: Medium–High **Affected Endpoints:** `GET /api/reports/{reportId}`, `GET /api/reports/user/{userId}`, `GET /api/reports/{reportId}/latest` **Affected Component:** Reports Controller

Description: Three report retrieval endpoints fetched records by primary key and returned them to any authenticated user without tenant verification. A tenant user from Institution A could access a full report — including document hashes, scan results, threat classifications, and Lighthouse CIDs — belonging to Institution B by supplying a valid report ID.

The `GET /api/reports/user/{userId}` endpoint additionally allowed any authenticated user to retrieve another user's reports by specifying a user ID, with no check that the target user belonged to the caller's tenant.

Risk: Report records contain sensitive academic document metadata including SHA-256 hashes, IPFS content identifiers, scan results, and student-linked data. Exposure of this data to unauthorized tenants constitutes a direct breach of inter-institutional confidentiality.

Remediation Applied:

- Added caller resolution and tenant ownership checks to all three endpoints
- `GET {reportId}` and `GET {reportId}/latest` compare the caller's `TenantId` against the report's `TenantId`; mismatches return `HTTP 403 Forbidden`
- `GET user/{userId}` compares the caller's `TenantId` against the target user's `TenantId`; mismatches return `HTTP 403 Forbidden`
- Admin users retain cross-tenant visibility as required by the platform design
- A null-safety guard was added to the user lookup in `GET user/{userId}` to return `HTTP 401 Unauthorized` for unresolvable caller identities

Files Modified:

- `VERACISMAPI/Controllers/ReportsController.cs`

VUL-05 — Missing Tenant Isolation on Encrypted File View and Download

Severity: Medium–High **Affected Endpoints:** `GET /api/verify/cid/{cid}`, `GET /api/verify/download/{cid}` **Affected Component:** Verify Controller

Description: The encrypted file view and download endpoints — which proxy decrypted document content from the Lighthouse/IPFS storage layer — authenticated callers but performed no tenant ownership check. Any authenticated tenant user could view or download any encrypted document from any institution by providing a valid CID, even if that document belonged to a different tenant.

This was distinct from the public verification endpoints (which confirm file existence and hash integrity) — those remain intentionally open by design, as document verification is a public-facing feature of the platform.

Risk: Actual document content — transcripts, credentials, and other academic records — could be retrieved by authenticated users from other institutions. This represents the most direct data confidentiality risk of all findings, as it exposes the underlying protected content rather than metadata.

Access control rules confirmed and applied:

User Type	View/Download	Public Verify
Admin	Any CID across all tenants	Allowed

User Type	View/Download	Public Verify
Tenant User	Own tenant's CIDs only	Allowed
Unauthenticated / Public	Not permitted	Allowed

Remediation Applied:

- Added a private `EnforceTenantOwnershipAsync(cid)` helper method to the Verify Controller
- The helper looks up the Report record by CID to determine its owning tenant, then resolves the authenticated caller's tenant identity from the user database
- Admin users (verified via JWT role claims) pass through without restriction
- Non-admin users receive `HTTP 403 Forbidden` if the document's `TenantId` does not match their own
- Both the view and download endpoints invoke this guard before any Lighthouse decryption is initiated
- All public verification endpoints (`public-check`, `info`, `log`, `deals`) remain unaffected

Files Modified:

- `VERACISMAPI/Controllers/VerifyController.cs`

Deferred Findings (Tracked for Next Sprint)

The following issues were identified during the assessment but are deferred pending further prioritization:

ID	Description	Rationale for Deferral
DEF-01	BOLA on <code>GET /api/reports/user/{userId}</code> — partial tenant check	Addressed in VUL-04; cross-user enumeration within a tenant requires additional UX review
DEF-02	Nullable reference warnings in <code>TenantRepository.cs</code> and <code>UsersController.cs</code>	Pre-existing warnings; no runtime impact confirmed. Scheduled for nullable annotation cleanup

Remediation Summary

Finding	Severity	Vectors Closed	Frontend Updated
VUL-01	Low	Pagination enforced server-side	Yes
VUL-02	Medium–High	Cross-tenant read + write paths	No (backend-only)
VUL-03	Low–Medium	Upload allowlist + UI restriction	Yes
VUL-04	Medium–High	Cross-tenant report read paths	No (backend-only)
VUL-05	Medium–High	Cross-tenant document view/download	No (backend-only)

Recommendations

1. **Penetration retest** — Conduct a targeted retest of all five remediated endpoints using the reproduction procedures in [VULNERABILITY_REPRODUCTION_RUNBOOK.md](#) to confirm closures
2. **Centralize authorization logic** — The repeated user-resolution and tenant-comparison pattern across controllers is a candidate for extraction into a shared base controller or authorization handler, reducing future risk of omission
3. **Address remaining BOLA surface** — [GET /api/reports](#) and verify controller endpoints remain partially reviewed. A full authorization audit of all remaining controller endpoints is recommended before the next production release
4. **Enable nullable reference types enforcement** — Resolve the pre-existing CS8601/CS8604/CS8605 warnings to eliminate potential null-dereference paths in production
5. **Rate limiting** — Upload endpoints currently have no rate limiting beyond file size. Implementing per-tenant upload rate limits is recommended to prevent abuse of the scanning pipeline

This report was prepared by the VERACISM development team. For questions or follow-up, contact the project technical lead.